

CIFAR-100 CNN Experiment: Local VS Code / Terminal Results Walkthrough

Deep Learning Course Assignment

This section documents a local terminal execution of the CIFAR-100 CNN experiment. The model was run as a standalone Python script from VS Code / terminal instead of inside a notebook. The purpose was to confirm that the same image classification workflow could run locally, save result plots as image files, and print training progress directly in the terminal.

Dataset: CIFAR-100

Framework: PyTorch

Execution: Local Python virtual environment through VS Code / terminal

Device: PyTorch CUDA GPU execution shown in terminal output

1. Purpose of this run

This run tested the CIFAR-100 CNN workflow as a standalone Python script. The script loaded the CIFAR-100 dataset, selected a smaller 10-class animal subset, normalized the images, converted the data into PyTorch tensors, trained a convolutional neural network, saved result plots, and printed the final test accuracy in the terminal.

This report is separate from the Google Colab notebook and the AMD Cloud / ROCm walkthrough. It focuses only on the local terminal runs and the results visible in the VS Code / terminal screenshots.

2. Execution environment

The script was executed from a Python virtual environment in the terminal. The terminal output repeatedly showed:

```
Training on: cuda
```

This confirms that the PyTorch script used GPU acceleration for these local runs. A warning from `torchvision.datasets.cifar.py` appeared several times:

```
VisibleDeprecationWarning: dtype(): align should be passed as Python or  
NumPy boolean
```

This warning did not stop the experiment. The CIFAR-100 files were already downloaded and verified, and the training process continued normally.

3. Full script used for the terminal run

The full script was run as `main.py`. It loads CIFAR-100, filters the dataset to animal classes, creates PyTorch tensors and DataLoaders, trains a CNN using LeakyReLU and Adam, saves plots, and prints the final test accuracy.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import torchvision
import numpy as np
import matplotlib.pyplot as plt

# Load CIFAR-100 dataset
trainset = torchvision.datasets.CIFAR100(root='./data', train=True,
download=True)
testset = torchvision.datasets.CIFAR100(root='./data', train=False,
download=True)

X_train = np.array(trainset.data)
y_train = np.array(trainset.targets)
X_test = np.array(testset.data)
y_test = np.array(testset.targets)

# Filter to 10 classes
selected_classes = [3, 21, 27, 31, 38, 42, 43, 88, 72, 97] # animal
classes
class_names = ['bear', 'chimpanzee', 'crocodile', 'elephant', 'kangaroo',
'leopard', 'lion', 'tiger', 'wolf', 'seal']

train_mask = np.isin(y_train, selected_classes)
X_train, y_train = X_train[train_mask], y_train[train_mask]

test_mask = np.isin(y_test, selected_classes)
X_test, y_test = X_test[test_mask], y_test[test_mask]

label_map = {orig: new for new, orig in enumerate(selected_classes)}
y_train = np.array([label_map[l] for l in y_train])
y_test = np.array([label_map[l] for l in y_test])

# Normalize pixel values to be between 0 and 1
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# Convert to PyTorch tensors
X_train_t = torch.tensor(X_train).permute(0, 3, 1, 2)
y_train_t = torch.tensor(y_train, dtype=torch.long)
X_test_t = torch.tensor(X_test).permute(0, 3, 1, 2)
y_test_t = torch.tensor(y_test, dtype=torch.long)
```

```

# Create DataLoaders
train_loader = DataLoader(TensorDataset(X_train_t, y_train_t),
batch_size=16, shuffle=True)
test_loader = DataLoader(TensorDataset(X_test_t, y_test_t), batch_size=16,
shuffle=False)

# Visualize some training images
plt.figure(figsize=(10, 10))
for i in range(16):
    plt.subplot(4, 4, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i])
    plt.xlabel(class_names[y_train[i]])
plt.savefig('sample_images.png')

# Build the CNN model
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, (3, 3), padding=1),
            nn.LeakyReLU(0.1),
            nn.Conv2d(32, 32, (3, 3), padding=1),
            nn.LeakyReLU(0.1),
            nn.MaxPool2d((2, 2)),
            nn.Dropout2d(0.25),
            nn.Conv2d(32, 64, (3, 3), padding=1),
            nn.LeakyReLU(0.1),
            nn.Conv2d(64, 64, (3, 3), padding=1),
            nn.LeakyReLU(0.1),
            nn.MaxPool2d((2, 2)),
            nn.Dropout2d(0.25),
            nn.Conv2d(64, 128, (3, 3), padding=1),
            nn.LeakyReLU(0.1),
            nn.Conv2d(128, 128, (3, 3), padding=1),
            nn.LeakyReLU(0.1),
            nn.MaxPool2d((2, 2)),
            nn.Dropout2d(0.25),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(2048, 512),
            nn.LeakyReLU(0.1),
            nn.Dropout(0.5),
            nn.Linear(512, 10),
        )

    def forward(self, x):

```

```

        x = self.features(x)
        x = self.classifier(x)
        return x

# Setup device, loss, optimizer
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = CNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

print(f"Training on: {device}")

# Train the model
EPOCHS = 30
train_accs, val_accs, train_losses, val_losses = [], [], [], []

for epoch in range(EPOCHS):
    model.train()
    correct, total, running_loss = 0, 0, 0.0

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        correct += (outputs.argmax(1) == y_batch).sum().item()
        total += y_batch.size(0)

    train_accs.append(correct / total)
    train_losses.append(running_loss / len(train_loader))

    model.eval()
    correct, total, running_loss = 0, 0, 0.0

    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            running_loss += loss.item()
            correct += (outputs.argmax(1) == y_batch).sum().item()
            total += y_batch.size(0)

    val_accs.append(correct / total)
    val_losses.append(running_loss / len(test_loader))

```

```

    print(f"Epoch {epoch+1}/{EPOCHS} | Train acc: {train_accs[-1]:.4f} |
Val acc: {val_accs[-1]:.4f}")

# Plot training & validation accuracy and loss values
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(train_accs, label='Train Accuracy')
plt.plot(val_accs, label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.savefig('results.png')
print("Plot saved to results.png")

# Predict on test images
def plot_predictions(index):
    model.eval()
    img_tensor = X_test_t[index].unsqueeze(0).to(device)
    with torch.no_grad():
        pred_probs = model(img_tensor)

    pred_label = class_names[pred_probs.argmax(1).item()]
    true_label = class_names[y_test[index]]

    plt.imshow(X_test[index])
    plt.title(f"True: {true_label} | Pred: {pred_label}")
    plt.axis('off')
    plt.savefig(f'prediction_{index}.png')

for i in range(5):
    plot_predictions(i)

print(f"\nFinal test accuracy: {val_accs[-1]*100:.2f}%")

```

4. Dataset setup

The script used CIFAR-100 and selected a 10-class animal subset: bear, chimpanzee, crocodile, elephant, kangaroo, leopard, lion, tiger, wolf, and seal.

The selected CIFAR-100 class indices were:

```
[3, 21, 27, 31, 38, 42, 43, 88, 72, 97]
```

The original CIFAR-100 labels were remapped into labels from 0 to 9 so the CNN could classify the selected subset correctly. The image pixel values were normalized to the range 0–1 before training.

The script also saved a sample image grid as:

```
sample_images.png
```

5. Model and training setup

The CNN used three convolutional blocks. Each block used convolutional layers, LeakyReLU activation, max pooling, and dropout. The classifier flattened the final feature maps and passed them through a dense layer before producing predictions for the 10 selected classes.

The main model configuration was:

Component	Setting
Model type	Convolutional Neural Network
Activation	LeakyReLU
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Batch size	16 in the full terminal script
Output classes	10
Saved plots	sample_images.png , results.png , prediction images

The script printed training and validation accuracy for each epoch in the terminal. It also saved output images to disk, including the sample image grid, accuracy/loss plots, and prediction examples.

6. Saved output images from the script

This section contains the image files generated by the Python script and pulled from the environment. These are not terminal screenshots. They are saved plot/image outputs created by the code.

6.1 Sample images

The script saved the selected animal-class sample grid as `sample_images.png`.

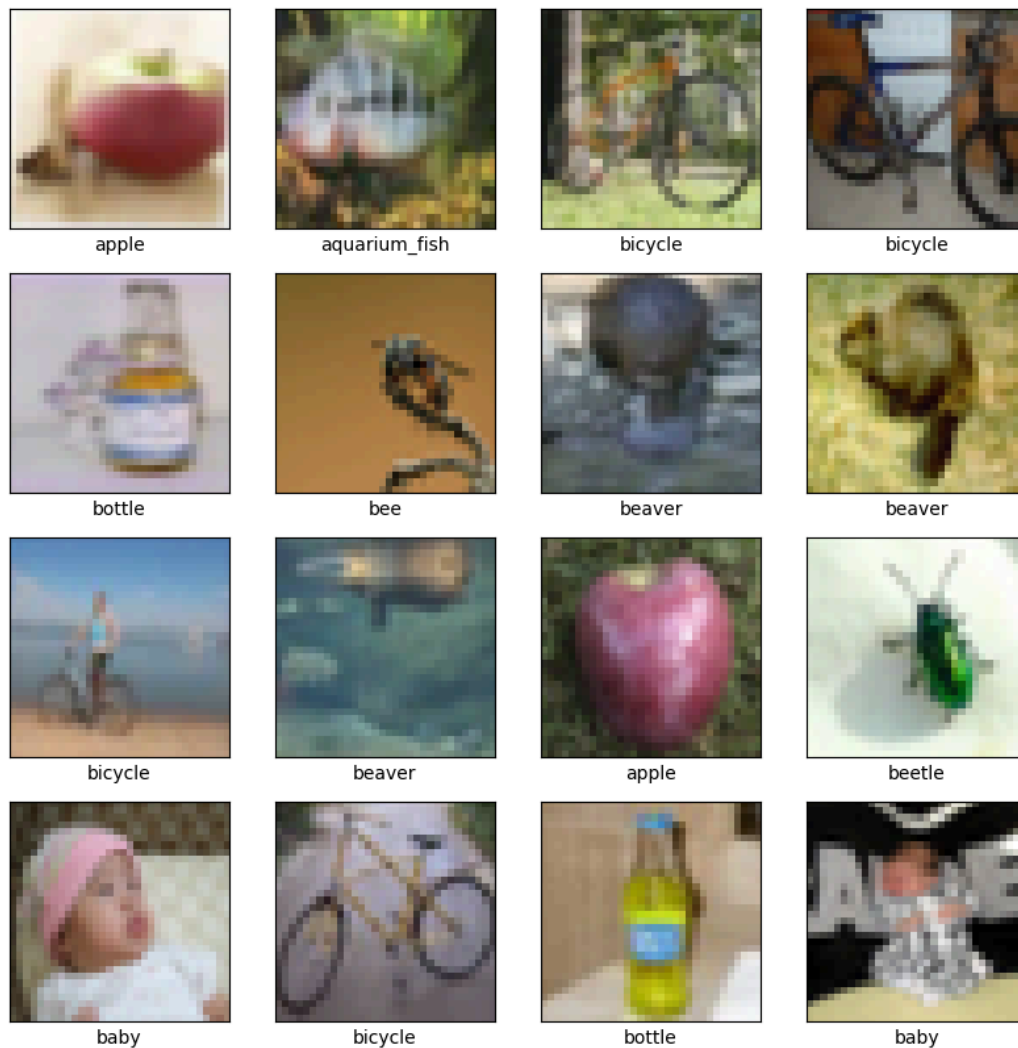


Figure 1. Sample images from the selected CIFAR-100 animal-class subset used in the local terminal run.

6.2 Training curves

The script saved training and validation accuracy/loss curves to image files such as `results.png`, `baseline_results.png`, and other run-specific result plots.

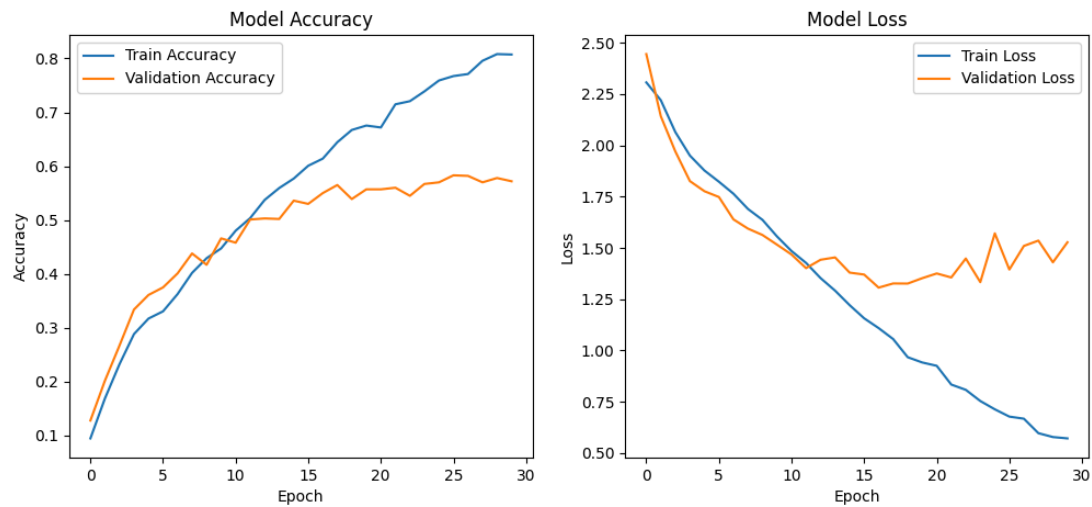


Figure 2. Saved training and validation accuracy/loss curves from the local terminal run, showing training progress and signs of overfitting.

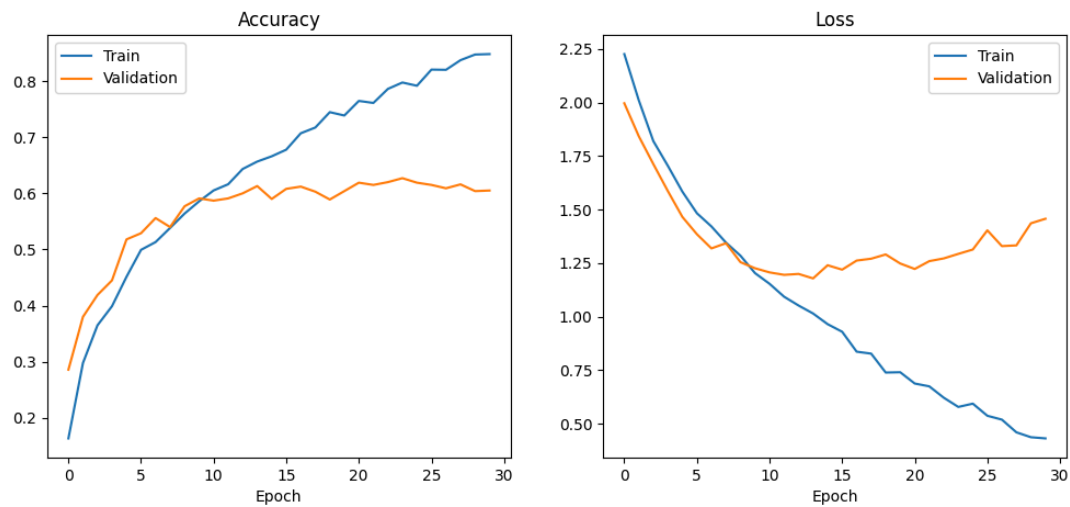


Figure 3. Saved baseline training and validation accuracy/loss curves from the local terminal run.

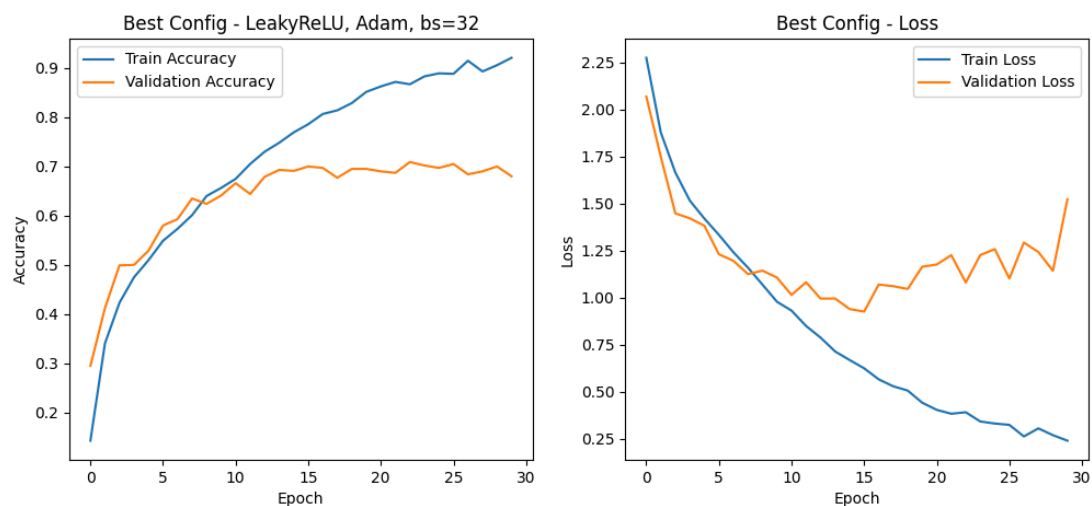


Figure 4. Saved best-configuration training curves using LeakyReLU, Adam, and batch size 32.

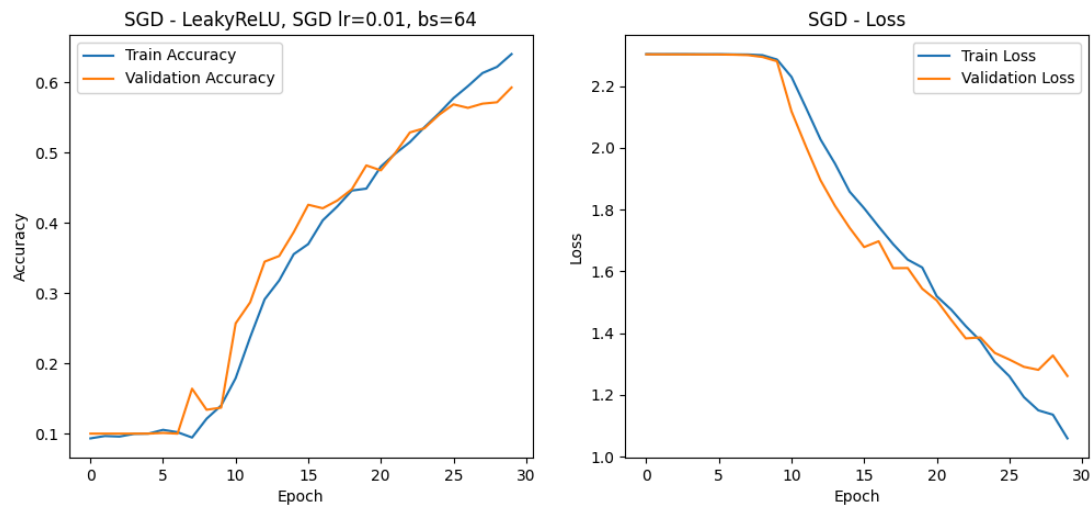


Figure 5. Saved SGD optimizer training curves using LeakyReLU, SGD learning rate 0.01, and batch size 64.

6.3 Prediction examples

The script saved prediction examples as separate image files. These show both correct and incorrect predictions on visually similar animal classes.

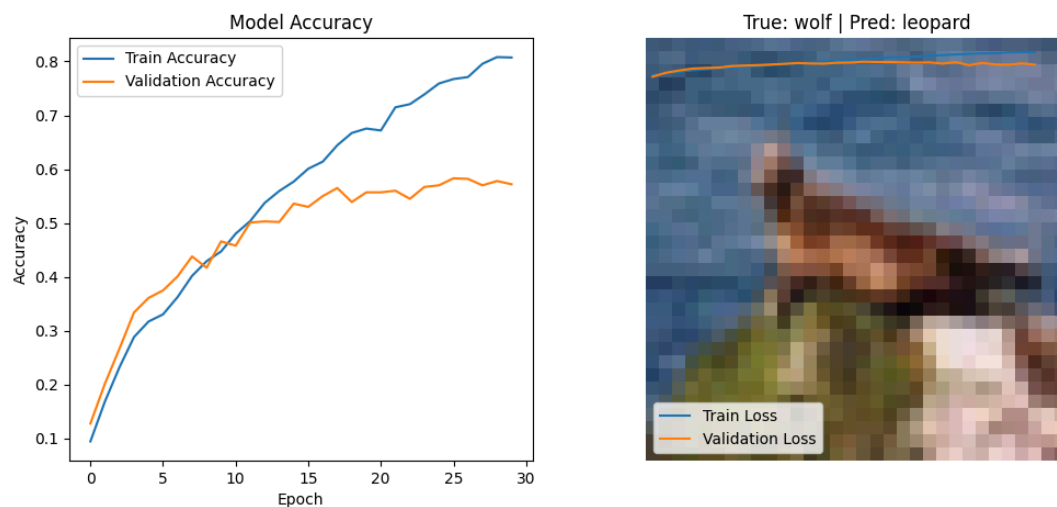


Figure 6a. Prediction example where the true class was wolf and the model predicted leopard.

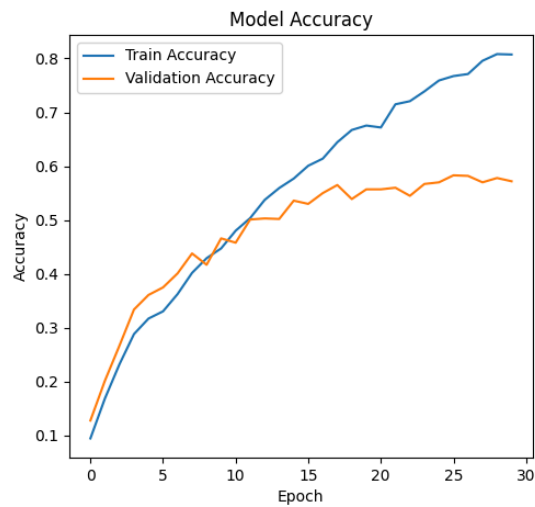


Figure 6b. Prediction example where the true class was lion and the model predicted leopard.

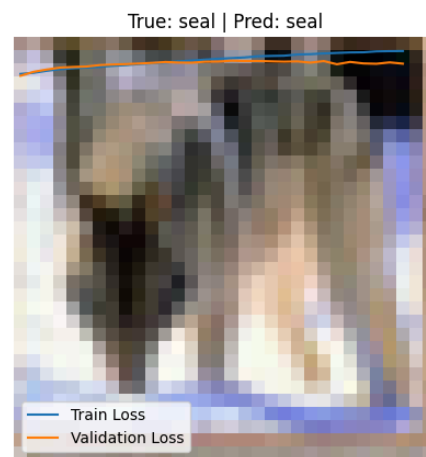
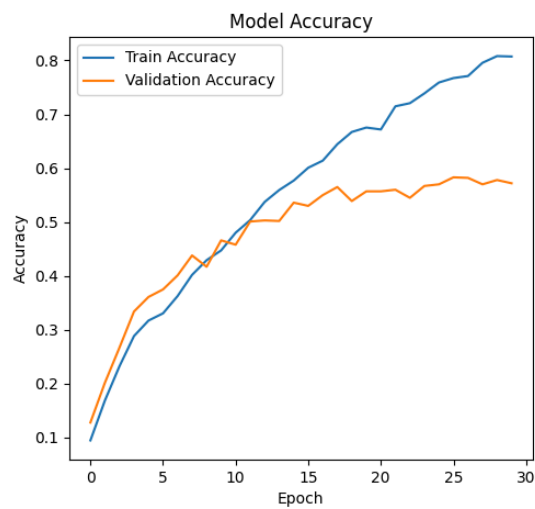


Figure 6c. Prediction example where the true class was seal and the model correctly predicted seal.

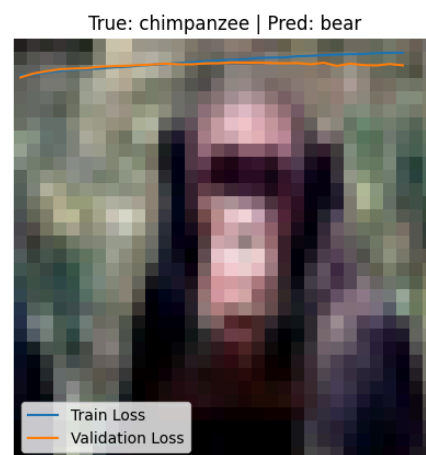
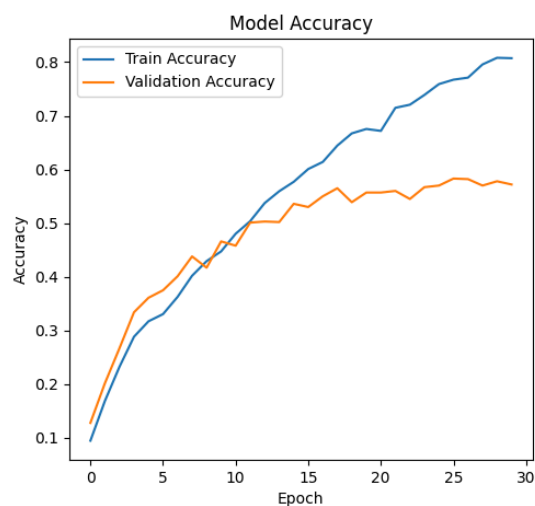


Figure 6d. Prediction example where the true class was chimpanzee and the model predicted bear.

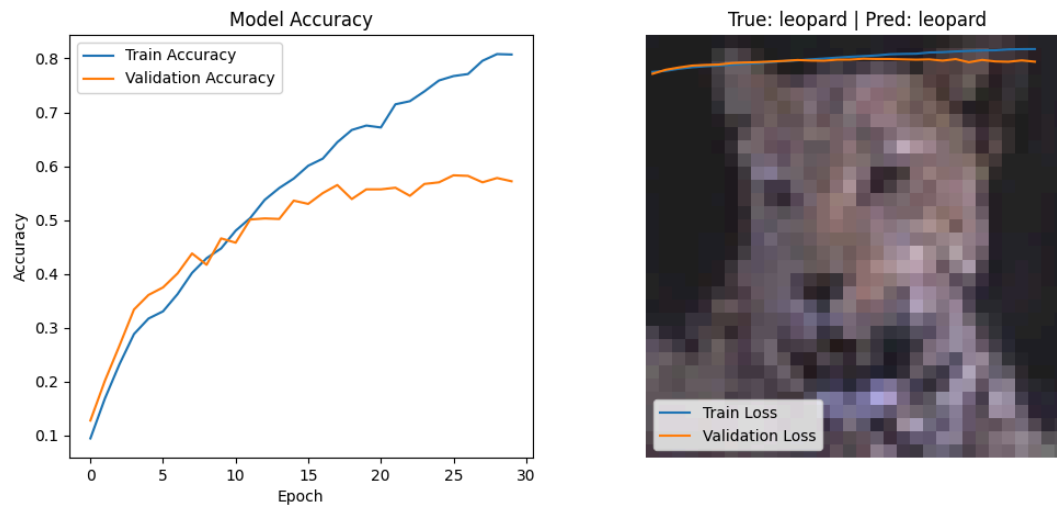


Figure 6e. Prediction example where the true class was leopard and the model correctly predicted leopard.

7. Terminal screenshot evidence

This section contains terminal screenshots. These screenshots are used as evidence that the script ran in the local terminal, used CUDA, printed epoch-by-epoch training progress, and produced the final test accuracy values.

```
Training on: cuda
```

Figure 7. Terminal output confirming that the local PyTorch script used CUDA GPU acceleration.

```
Training on: cddc
Epoch 1/30 | Train acc: 0.1492 | Val acc: 0.2460
Epoch 2/30 | Train acc: 0.2564 | Val acc: 0.3380
Epoch 3/30 | Train acc: 0.3648 | Val acc: 0.3980
Epoch 4/30 | Train acc: 0.4156 | Val acc: 0.4510
Epoch 5/30 | Train acc: 0.4492 | Val acc: 0.5270
Epoch 6/30 | Train acc: 0.4850 | Val acc: 0.4940
Epoch 7/30 | Train acc: 0.5062 | Val acc: 0.5290
Epoch 8/30 | Train acc: 0.5258 | Val acc: 0.5630
Epoch 9/30 | Train acc: 0.5456 | Val acc: 0.5850
Epoch 10/30 | Train acc: 0.5590 | Val acc: 0.5710
Epoch 11/30 | Train acc: 0.5784 | Val acc: 0.6030
Epoch 12/30 | Train acc: 0.5990 | Val acc: 0.6000
Epoch 13/30 | Train acc: 0.6172 | Val acc: 0.6180
Epoch 14/30 | Train acc: 0.6288 | Val acc: 0.5810
Epoch 15/30 | Train acc: 0.6376 | Val acc: 0.5910
Epoch 16/30 | Train acc: 0.6602 | Val acc: 0.6030
Epoch 17/30 | Train acc: 0.6740 | Val acc: 0.5960
Epoch 18/30 | Train acc: 0.6914 | Val acc: 0.6060
Epoch 19/30 | Train acc: 0.7106 | Val acc: 0.6050
Epoch 20/30 | Train acc: 0.7224 | Val acc: 0.6010
Epoch 21/30 | Train acc: 0.7286 | Val acc: 0.6240
Epoch 22/30 | Train acc: 0.7488 | Val acc: 0.6210
Epoch 23/30 | Train acc: 0.7576 | Val acc: 0.6330
Epoch 24/30 | Train acc: 0.7816 | Val acc: 0.6190
Epoch 25/30 | Train acc: 0.7734 | Val acc: 0.6110
Epoch 26/30 | Train acc: 0.7972 | Val acc: 0.6280
Epoch 27/30 | Train acc: 0.8074 | Val acc: 0.6250
Epoch 28/30 | Train acc: 0.8222 | Val acc: 0.6190
Epoch 29/30 | Train acc: 0.8166 | Val acc: 0.6300
Epoch 30/30 | Train acc: 0.8402 | Val acc: 0.6150
Plot saved to baseline_results.png

Final test accuracy: 61.50%
```

Figure 8. Baseline-style terminal run reaching **61.50%** final test accuracy.

```
Epoch 1/30 | Train acc: 0.1636 | Val acc: 0.2860
Epoch 2/30 | Train acc: 0.2978 | Val acc: 0.3800
Epoch 3/30 | Train acc: 0.3650 | Val acc: 0.4190
Epoch 4/30 | Train acc: 0.3990 | Val acc: 0.4450
Epoch 5/30 | Train acc: 0.4514 | Val acc: 0.5180
Epoch 6/30 | Train acc: 0.4994 | Val acc: 0.5290
Epoch 7/30 | Train acc: 0.5132 | Val acc: 0.5560
Epoch 8/30 | Train acc: 0.5384 | Val acc: 0.5400
Epoch 9/30 | Train acc: 0.5640 | Val acc: 0.5770
Epoch 10/30 | Train acc: 0.5858 | Val acc: 0.5910
Epoch 11/30 | Train acc: 0.6052 | Val acc: 0.5870
Epoch 12/30 | Train acc: 0.6162 | Val acc: 0.5910
Epoch 13/30 | Train acc: 0.6434 | Val acc: 0.6000
Epoch 14/30 | Train acc: 0.6566 | Val acc: 0.6130
Epoch 15/30 | Train acc: 0.6660 | Val acc: 0.5900
Epoch 16/30 | Train acc: 0.6778 | Val acc: 0.6080
Epoch 17/30 | Train acc: 0.7070 | Val acc: 0.6120
Epoch 18/30 | Train acc: 0.7172 | Val acc: 0.6030
Epoch 19/30 | Train acc: 0.7446 | Val acc: 0.5890
Epoch 20/30 | Train acc: 0.7386 | Val acc: 0.6040
Epoch 21/30 | Train acc: 0.7646 | Val acc: 0.6190
Epoch 22/30 | Train acc: 0.7610 | Val acc: 0.6150
Epoch 23/30 | Train acc: 0.7860 | Val acc: 0.6200
Epoch 24/30 | Train acc: 0.7974 | Val acc: 0.6270
Epoch 25/30 | Train acc: 0.7916 | Val acc: 0.6190
Epoch 26/30 | Train acc: 0.8204 | Val acc: 0.6150
Epoch 27/30 | Train acc: 0.8200 | Val acc: 0.6090
Epoch 28/30 | Train acc: 0.8372 | Val acc: 0.6160
Epoch 29/30 | Train acc: 0.8472 | Val acc: 0.6040
Epoch 30/30 | Train acc: 0.8480 | Val acc: 0.6050
Plot saved to baseline_results.png

Final test accuracy: 60.50%
```

Figure 9. Additional baseline-style terminal run reaching **60.50%** final test accuracy.

Training on 10000			
Epoch 1/50	Train acc: 0.1116	Val acc: 0.1930	
Epoch 2/50	Train acc: 0.1776	Val acc: 0.1660	
Epoch 3/50	Train acc: 0.2260	Val acc: 0.2810	
Epoch 4/50	Train acc: 0.2818	Val acc: 0.3410	
Epoch 5/50	Train acc: 0.3298	Val acc: 0.3560	
Epoch 6/50	Train acc: 0.3520	Val acc: 0.3850	
Epoch 7/50	Train acc: 0.3994	Val acc: 0.4080	
Epoch 8/50	Train acc: 0.4094	Val acc: 0.4290	
Epoch 9/50	Train acc: 0.4436	Val acc: 0.4720	
Epoch 10/50	Train acc: 0.4638	Val acc: 0.4600	
Epoch 11/50	Train acc: 0.4832	Val acc: 0.4930	
Epoch 12/50	Train acc: 0.4906	Val acc: 0.5140	
Epoch 13/50	Train acc: 0.5116	Val acc: 0.5200	
Epoch 14/50	Train acc: 0.5284	Val acc: 0.5240	
Epoch 15/50	Train acc: 0.5442	Val acc: 0.5390	
Epoch 16/50	Train acc: 0.5556	Val acc: 0.5370	
Epoch 17/50	Train acc: 0.5848	Val acc: 0.5490	
Epoch 18/50	Train acc: 0.5870	Val acc: 0.5520	
Epoch 19/50	Train acc: 0.5980	Val acc: 0.5620	
Epoch 20/50	Train acc: 0.6202	Val acc: 0.5690	
Epoch 21/50	Train acc: 0.6350	Val acc: 0.5590	
Epoch 22/50	Train acc: 0.6430	Val acc: 0.5700	
Epoch 23/50	Train acc: 0.6554	Val acc: 0.5480	
Epoch 24/50	Train acc: 0.6766	Val acc: 0.5600	
Epoch 25/50	Train acc: 0.6736	Val acc: 0.5620	

```
Epoch 24/50 | Train acc: 0.7088 | Val acc: 0.5600
Epoch 25/50 | Train acc: 0.6736 | Val acc: 0.5620
Epoch 26/50 | Train acc: 0.6778 | Val acc: 0.5760
Epoch 27/50 | Train acc: 0.7216 | Val acc: 0.5670
Epoch 28/50 | Train acc: 0.7260 | Val acc: 0.5650
Epoch 29/50 | Train acc: 0.7334 | Val acc: 0.5590
Epoch 30/50 | Train acc: 0.7488 | Val acc: 0.5700
Epoch 31/50 | Train acc: 0.7618 | Val acc: 0.5560
Epoch 32/50 | Train acc: 0.7826 | Val acc: 0.5760
Epoch 33/50 | Train acc: 0.7844 | Val acc: 0.5660
Epoch 34/50 | Train acc: 0.7970 | Val acc: 0.5620
Epoch 35/50 | Train acc: 0.7974 | Val acc: 0.5420
Epoch 36/50 | Train acc: 0.8064 | Val acc: 0.5470
Epoch 37/50 | Train acc: 0.8370 | Val acc: 0.5520
Epoch 38/50 | Train acc: 0.8196 | Val acc: 0.5530
Epoch 39/50 | Train acc: 0.8508 | Val acc: 0.5720
Epoch 40/50 | Train acc: 0.8362 | Val acc: 0.5450
Epoch 41/50 | Train acc: 0.8426 | Val acc: 0.5520
Epoch 42/50 | Train acc: 0.8640 | Val acc: 0.5510
Epoch 43/50 | Train acc: 0.8582 | Val acc: 0.5720
Epoch 44/50 | Train acc: 0.8784 | Val acc: 0.5610
Epoch 45/50 | Train acc: 0.8816 | Val acc: 0.5530
Epoch 46/50 | Train acc: 0.8776 | Val acc: 0.5570
Epoch 47/50 | Train acc: 0.8818 | Val acc: 0.5370
Epoch 48/50 | Train acc: 0.8778 | Val acc: 0.5590
Epoch 49/50 | Train acc: 0.8860 | Val acc: 0.5550
Epoch 50/50 | Train acc: 0.8896 | Val acc: 0.5490
Plot saved to results.png

Final test accuracy: 54.90%
```

Figure 10. Deeper 50-epoch terminal run reaching **54.90%** final test accuracy.


```
Training on: c1000
Epoch 1/30 | Train acc: 0.1950 | Val acc: 0.3090
Epoch 2/30 | Train acc: 0.3446 | Val acc: 0.4320
Epoch 3/30 | Train acc: 0.4334 | Val acc: 0.5150
Epoch 4/30 | Train acc: 0.4930 | Val acc: 0.5090
Epoch 5/30 | Train acc: 0.5386 | Val acc: 0.5610
Epoch 6/30 | Train acc: 0.5714 | Val acc: 0.5670
Epoch 7/30 | Train acc: 0.5990 | Val acc: 0.6060
Epoch 8/30 | Train acc: 0.6270 | Val acc: 0.6130
Epoch 9/30 | Train acc: 0.6720 | Val acc: 0.6310
Epoch 10/30 | Train acc: 0.6736 | Val acc: 0.6560
Epoch 11/30 | Train acc: 0.7062 | Val acc: 0.6320
Epoch 12/30 | Train acc: 0.7258 | Val acc: 0.6610
Epoch 13/30 | Train acc: 0.7494 | Val acc: 0.6710
Epoch 14/30 | Train acc: 0.7674 | Val acc: 0.6600
Epoch 15/30 | Train acc: 0.7952 | Val acc: 0.6660
Epoch 16/30 | Train acc: 0.8066 | Val acc: 0.6820
Epoch 17/30 | Train acc: 0.8184 | Val acc: 0.6790
Epoch 18/30 | Train acc: 0.8438 | Val acc: 0.6760
Epoch 19/30 | Train acc: 0.8472 | Val acc: 0.6630
Epoch 20/30 | Train acc: 0.8638 | Val acc: 0.6670
Epoch 21/30 | Train acc: 0.8846 | Val acc: 0.6850
Epoch 22/30 | Train acc: 0.8928 | Val acc: 0.6860
Epoch 23/30 | Train acc: 0.8996 | Val acc: 0.6860
Epoch 24/30 | Train acc: 0.9026 | Val acc: 0.6930
Epoch 25/30 | Train acc: 0.8958 | Val acc: 0.6760
Epoch 26/30 | Train acc: 0.9166 | Val acc: 0.6800
Epoch 27/30 | Train acc: 0.9228 | Val acc: 0.6780
Epoch 28/30 | Train acc: 0.9192 | Val acc: 0.6890
Epoch 29/30 | Train acc: 0.9206 | Val acc: 0.6710
Epoch 30/30 | Train acc: 0.9312 | Val acc: 0.6730
Plot saved to results.png

Final test accuracy: 67.30%
(your) root@2:~/deep-learning#
```

Figure 11 LeakyReLU + Adam terminal run reaching 67.30% final test accuracy.**


```
Epoch 1/30 | Train acc: 0.0946 | Val acc: 0.1000
Epoch 2/30 | Train acc: 0.0974 | Val acc: 0.1000
Epoch 3/30 | Train acc: 0.0880 | Val acc: 0.1000
Epoch 4/30 | Train acc: 0.0992 | Val acc: 0.1000
Epoch 5/30 | Train acc: 0.1016 | Val acc: 0.1180
Epoch 6/30 | Train acc: 0.1080 | Val acc: 0.1020
Epoch 7/30 | Train acc: 0.1164 | Val acc: 0.1600
Epoch 8/30 | Train acc: 0.1322 | Val acc: 0.1890
Epoch 9/30 | Train acc: 0.1704 | Val acc: 0.2310
Epoch 10/30 | Train acc: 0.2284 | Val acc: 0.2660
Epoch 11/30 | Train acc: 0.2650 | Val acc: 0.2990
Epoch 12/30 | Train acc: 0.2942 | Val acc: 0.3620
Epoch 13/30 | Train acc: 0.3302 | Val acc: 0.3870
Epoch 14/30 | Train acc: 0.3564 | Val acc: 0.4010
Epoch 15/30 | Train acc: 0.3718 | Val acc: 0.3850
Epoch 16/30 | Train acc: 0.4136 | Val acc: 0.4420
Epoch 17/30 | Train acc: 0.4268 | Val acc: 0.4500
Epoch 18/30 | Train acc: 0.4440 | Val acc: 0.4730
Epoch 19/30 | Train acc: 0.4472 | Val acc: 0.4710
Epoch 20/30 | Train acc: 0.4778 | Val acc: 0.4930
Epoch 21/30 | Train acc: 0.4850 | Val acc: 0.4740
Epoch 22/30 | Train acc: 0.5066 | Val acc: 0.5060
Epoch 23/30 | Train acc: 0.5220 | Val acc: 0.4820
Epoch 24/30 | Train acc: 0.5398 | Val acc: 0.5200
Epoch 25/30 | Train acc: 0.5516 | Val acc: 0.5310
Epoch 26/30 | Train acc: 0.5744 | Val acc: 0.5310
Epoch 27/30 | Train acc: 0.5932 | Val acc: 0.5170
Epoch 28/30 | Train acc: 0.5946 | Val acc: 0.5510
Epoch 29/30 | Train acc: 0.6118 | Val acc: 0.5380
Epoch 30/30 | Train acc: 0.6162 | Val acc: 0.5530
Plot saved to results.png

Final test accuracy: 55.30%
```

Figure 12. SGD optimizer terminal run reaching **55.30%** final test accuracy after 30 epochs.

Epoch 1/80		Train acc: 0.0912		Val acc: 0.1000
Epoch 2/80		Train acc: 0.0958		Val acc: 0.1000
Epoch 3/80		Train acc: 0.0962		Val acc: 0.1000
Epoch 4/80		Train acc: 0.0936		Val acc: 0.1000
Epoch 5/80		Train acc: 0.1048		Val acc: 0.1540
Epoch 6/80		Train acc: 0.0940		Val acc: 0.1000
Epoch 7/80		Train acc: 0.1028		Val acc: 0.1010
Epoch 8/80		Train acc: 0.1090		Val acc: 0.1460
Epoch 9/80		Train acc: 0.1114		Val acc: 0.1270
Epoch 10/80		Train acc: 0.1396		Val acc: 0.1990
Epoch 11/80		Train acc: 0.2016		Val acc: 0.2220
Epoch 12/80		Train acc: 0.2438		Val acc: 0.2920
Epoch 13/80		Train acc: 0.2818		Val acc: 0.3390
Epoch 14/80		Train acc: 0.3380		Val acc: 0.3790
Epoch 15/80		Train acc: 0.3698		Val acc: 0.4040
Epoch 16/80		Train acc: 0.3848		Val acc: 0.4260
Epoch 17/80		Train acc: 0.4086		Val acc: 0.4570
Epoch 18/80		Train acc: 0.4340		Val acc: 0.4420
Epoch 19/80		Train acc: 0.4512		Val acc: 0.4590
Epoch 20/80		Train acc: 0.4624		Val acc: 0.4950
Epoch 40/80		Train acc: 0.7900		Val acc: 0.5950
Epoch 41/80		Train acc: 0.8138		Val acc: 0.5760
Epoch 42/80		Train acc: 0.8066		Val acc: 0.5800
Epoch 43/80		Train acc: 0.8316		Val acc: 0.5710
Epoch 44/80		Train acc: 0.8400		Val acc: 0.5760
Epoch 45/80		Train acc: 0.8474		Val acc: 0.5700
Epoch 46/80		Train acc: 0.8596		Val acc: 0.5650
Epoch 47/80		Train acc: 0.8560		Val acc: 0.5770
Epoch 48/80		Train acc: 0.8680		Val acc: 0.5870
Epoch 49/80		Train acc: 0.8684		Val acc: 0.5690
Epoch 50/80		Train acc: 0.8794		Val acc: 0.5820
Epoch 51/80		Train acc: 0.8916		Val acc: 0.5930
Epoch 52/80		Train acc: 0.8956		Val acc: 0.5980
Epoch 53/80		Train acc: 0.8958		Val acc: 0.5690
Epoch 54/80		Train acc: 0.8974		Val acc: 0.5830
Epoch 55/80		Train acc: 0.9060		Val acc: 0.5730
Epoch 56/80		Train acc: 0.8932		Val acc: 0.5870
Epoch 57/80		Train acc: 0.9182		Val acc: 0.5750
Epoch 58/80		Train acc: 0.9186		Val acc: 0.5790
Epoch 59/80		Train acc: 0.9138		Val acc: 0.5850
Epoch 60/80		Train acc: 0.9310		Val acc: 0.5920
Epoch 61/80		Train acc: 0.9444		Val acc: 0.5990

```
Epoch 59/80 | Train acc: 0.9310 | Val acc: 0.5920
Epoch 60/80 | Train acc: 0.9310 | Val acc: 0.5920
Epoch 61/80 | Train acc: 0.9144 | Val acc: 0.5800
Epoch 62/80 | Train acc: 0.9374 | Val acc: 0.5940
Epoch 63/80 | Train acc: 0.9296 | Val acc: 0.5920
Epoch 64/80 | Train acc: 0.9316 | Val acc: 0.5800
Epoch 65/80 | Train acc: 0.9062 | Val acc: 0.5840
Epoch 66/80 | Train acc: 0.9358 | Val acc: 0.5740
Epoch 67/80 | Train acc: 0.9452 | Val acc: 0.5960
Epoch 68/80 | Train acc: 0.9340 | Val acc: 0.5820
Epoch 69/80 | Train acc: 0.9438 | Val acc: 0.5800
Epoch 70/80 | Train acc: 0.9416 | Val acc: 0.5950
Epoch 71/80 | Train acc: 0.9474 | Val acc: 0.5970
Epoch 72/80 | Train acc: 0.9492 | Val acc: 0.5920
Epoch 73/80 | Train acc: 0.9490 | Val acc: 0.5900
Epoch 74/80 | Train acc: 0.9526 | Val acc: 0.5880
Epoch 75/80 | Train acc: 0.9518 | Val acc: 0.5880
Epoch 76/80 | Train acc: 0.9564 | Val acc: 0.5860
Epoch 77/80 | Train acc: 0.9506 | Val acc: 0.5870
Epoch 78/80 | Train acc: 0.9446 | Val acc: 0.5920
Epoch 79/80 | Train acc: 0.9552 | Val acc: 0.5850
Epoch 80/80 | Train acc: 0.9578 | Val acc: 0.5920
Plot saved to results.png

Final test accuracy: 59.20%
(venv) root@2:~/deep-learning#
```

Figure 13. Extended SGD optimizer terminal run reaching **59.20%** final test accuracy after 80 epochs.

```
Epoch 1/30 | Train acc: 0.1332 | Val acc: 0.2230
Epoch 2/30 | Train acc: 0.2556 | Val acc: 0.3430
Epoch 3/30 | Train acc: 0.3668 | Val acc: 0.4520
Epoch 4/30 | Train acc: 0.4400 | Val acc: 0.4900
Epoch 5/30 | Train acc: 0.4950 | Val acc: 0.5180
Epoch 6/30 | Train acc: 0.5254 | Val acc: 0.5660
Epoch 7/30 | Train acc: 0.5522 | Val acc: 0.5730
Epoch 8/30 | Train acc: 0.5762 | Val acc: 0.6190
Epoch 9/30 | Train acc: 0.6068 | Val acc: 0.6090
Epoch 10/30 | Train acc: 0.6336 | Val acc: 0.6260
Epoch 11/30 | Train acc: 0.6666 | Val acc: 0.6260
Epoch 12/30 | Train acc: 0.6806 | Val acc: 0.6310
Epoch 13/30 | Train acc: 0.6856 | Val acc: 0.6380
Epoch 14/30 | Train acc: 0.7036 | Val acc: 0.6660
Epoch 15/30 | Train acc: 0.7394 | Val acc: 0.6560
Epoch 16/30 | Train acc: 0.7568 | Val acc: 0.6570
Epoch 17/30 | Train acc: 0.7774 | Val acc: 0.6620
Epoch 18/30 | Train acc: 0.7844 | Val acc: 0.6610
Epoch 19/30 | Train acc: 0.7994 | Val acc: 0.6670
Epoch 20/30 | Train acc: 0.8174 | Val acc: 0.6550
Epoch 21/30 | Train acc: 0.8184 | Val acc: 0.6790
Epoch 22/30 | Train acc: 0.8210 | Val acc: 0.6830
Epoch 23/30 | Train acc: 0.8530 | Val acc: 0.6810
Epoch 24/30 | Train acc: 0.8560 | Val acc: 0.6840
Epoch 25/30 | Train acc: 0.8710 | Val acc: 0.6890
Epoch 26/30 | Train acc: 0.8912 | Val acc: 0.6650
Epoch 27/30 | Train acc: 0.8842 | Val acc: 0.6780
Epoch 28/30 | Train acc: 0.8828 | Val acc: 0.6750
Epoch 29/30 | Train acc: 0.8960 | Val acc: 0.6750
Epoch 30/30 | Train acc: 0.8580 | Val acc: 0.6800
Plot saved to results.png

Final test accuracy: 68.00%
```

Figure 14. Tuned terminal run reaching **68.00%** final test accuracy.

Training on Case

Epoch 1/50	Train acc: 0.1850	Val acc: 0.3470
Epoch 2/50	Train acc: 0.3736	Val acc: 0.4860
Epoch 3/50	Train acc: 0.4458	Val acc: 0.5120
Epoch 4/50	Train acc: 0.4982	Val acc: 0.5280
Epoch 5/50	Train acc: 0.5486	Val acc: 0.5900
Epoch 6/50	Train acc: 0.5848	Val acc: 0.6260
Epoch 7/50	Train acc: 0.6182	Val acc: 0.6490
Epoch 8/50	Train acc: 0.6448	Val acc: 0.6380
Epoch 9/50	Train acc: 0.6732	Val acc: 0.6330
Epoch 10/50	Train acc: 0.6940	Val acc: 0.6630
Epoch 11/50	Train acc: 0.7276	Val acc: 0.6900
Epoch 12/50	Train acc: 0.7552	Val acc: 0.6550
Epoch 13/50	Train acc: 0.7686	Val acc: 0.6670
Epoch 14/50	Train acc: 0.7850	Val acc: 0.6710
Epoch 15/50	Train acc: 0.8050	Val acc: 0.6910
Epoch 16/50	Train acc: 0.8356	Val acc: 0.6680
Epoch 17/50	Train acc: 0.8352	Val acc: 0.6840
Epoch 18/50	Train acc: 0.8544	Val acc: 0.6840
Epoch 19/50	Train acc: 0.8626	Val acc: 0.7060
Epoch 20/50	Train acc: 0.8700	Val acc: 0.6960
Epoch 21/50	Train acc: 0.8770	Val acc: 0.7060
Epoch 22/50	Train acc: 0.8862	Val acc: 0.7030
Epoch 23/50	Train acc: 0.8818	Val acc: 0.6710
Epoch 24/50	Train acc: 0.8902	Val acc: 0.6850
Epoch 25/50	Train acc: 0.9088	Val acc: 0.7120

Epoch 26/50	Train acc: 0.9188	Val acc: 0.7220
-------------	-------------------	-----------------

```
Epoch 26/50 | Train acc: 0.9120 | Val acc: 0.7080
Epoch 27/50 | Train acc: 0.9076 | Val acc: 0.7070
Epoch 28/50 | Train acc: 0.9184 | Val acc: 0.7030
Epoch 29/50 | Train acc: 0.9180 | Val acc: 0.6930
Epoch 30/50 | Train acc: 0.9262 | Val acc: 0.6960
Epoch 31/50 | Train acc: 0.9264 | Val acc: 0.6940
Epoch 32/50 | Train acc: 0.9280 | Val acc: 0.7170
Epoch 33/50 | Train acc: 0.9258 | Val acc: 0.6890
Epoch 34/50 | Train acc: 0.9262 | Val acc: 0.7100
Epoch 35/50 | Train acc: 0.9252 | Val acc: 0.6780
Epoch 36/50 | Train acc: 0.9310 | Val acc: 0.7010
Epoch 37/50 | Train acc: 0.9316 | Val acc: 0.7030
Epoch 38/50 | Train acc: 0.9288 | Val acc: 0.6940
Epoch 39/50 | Train acc: 0.9370 | Val acc: 0.6900
Epoch 40/50 | Train acc: 0.9424 | Val acc: 0.6900
Epoch 41/50 | Train acc: 0.9454 | Val acc: 0.7170
Epoch 42/50 | Train acc: 0.9362 | Val acc: 0.6980
Epoch 43/50 | Train acc: 0.9348 | Val acc: 0.6980
Epoch 44/50 | Train acc: 0.9462 | Val acc: 0.6950
Epoch 45/50 | Train acc: 0.9336 | Val acc: 0.7110
Epoch 46/50 | Train acc: 0.9500 | Val acc: 0.7150
Epoch 47/50 | Train acc: 0.9460 | Val acc: 0.7010
Epoch 48/50 | Train acc: 0.9452 | Val acc: 0.6840
Epoch 49/50 | Train acc: 0.9340 | Val acc: 0.6990
Epoch 50/50 | Train acc: 0.9368 | Val acc: 0.6990
Plot saved to results.png

Final test accuracy: 69.90%
(your) test02: /deep-learning#
```

Figure 15. Best visible local terminal result, reaching **69.90%** final test accuracy.

```
Training on: cuda
Epoch 1/15 | Train acc: 0.2120 | Val acc: 0.3940
Epoch 2/15 | Train acc: 0.3712 | Val acc: 0.4420
Epoch 3/15 | Train acc: 0.4440 | Val acc: 0.5120
Epoch 4/15 | Train acc: 0.4892 | Val acc: 0.5490
Epoch 5/15 | Train acc: 0.5166 | Val acc: 0.5620
Epoch 6/15 | Train acc: 0.5492 | Val acc: 0.5820
Epoch 7/15 | Train acc: 0.5938 | Val acc: 0.6040
Epoch 8/15 | Train acc: 0.6138 | Val acc: 0.6060
Epoch 9/15 | Train acc: 0.6464 | Val acc: 0.6410
Epoch 10/15 | Train acc: 0.6774 | Val acc: 0.6680
Epoch 11/15 | Train acc: 0.7022 | Val acc: 0.6480
Epoch 12/15 | Train acc: 0.7114 | Val acc: 0.6510
Epoch 13/15 | Train acc: 0.7412 | Val acc: 0.6820
Epoch 14/15 | Train acc: 0.7534 | Val acc: 0.6580
Epoch 15/15 | Train acc: 0.7702 | Val acc: 0.6580
Plot saved to results.png

Final test accuracy: 65.80%
(venv) root@2:~/deep-learning#
```

Figure 16. Shorter 15-epoch terminal run reaching **65.80%** final test accuracy.

```

Training on: c000
Epoch 1/30 | Train acc: 0.1270 | Val acc: 0.1670
Epoch 2/30 | Train acc: 0.2216 | Val acc: 0.3120
Epoch 3/30 | Train acc: 0.2908 | Val acc: 0.3190
Epoch 4/30 | Train acc: 0.3388 | Val acc: 0.3930
Epoch 5/30 | Train acc: 0.3884 | Val acc: 0.3990
Epoch 6/30 | Train acc: 0.4256 | Val acc: 0.4230
Epoch 7/30 | Train acc: 0.4600 | Val acc: 0.4610
Epoch 8/30 | Train acc: 0.4878 | Val acc: 0.4620
Epoch 9/30 | Train acc: 0.5158 | Val acc: 0.5090
Epoch 10/30 | Train acc: 0.5462 | Val acc: 0.5230
Epoch 11/30 | Train acc: 0.5740 | Val acc: 0.5240
Epoch 12/30 | Train acc: 0.5982 | Val acc: 0.5300
Epoch 13/30 | Train acc: 0.6204 | Val acc: 0.5550
Epoch 14/30 | Train acc: 0.6528 | Val acc: 0.5700
Epoch 15/30 | Train acc: 0.6752 | Val acc: 0.5610
Epoch 16/30 | Train acc: 0.6952 | Val acc: 0.5620
Epoch 17/30 | Train acc: 0.7306 | Val acc: 0.5590
Epoch 18/30 | Train acc: 0.7372 | Val acc: 0.5710
Epoch 19/30 | Train acc: 0.7606 | Val acc: 0.5750
Epoch 20/30 | Train acc: 0.7800 | Val acc: 0.5700
Epoch 21/30 | Train acc: 0.7904 | Val acc: 0.5660
Epoch 22/30 | Train acc: 0.8086 | Val acc: 0.5770
Epoch 23/30 | Train acc: 0.8262 | Val acc: 0.5780
Epoch 24/30 | Train acc: 0.8190 | Val acc: 0.5630
Epoch 25/30 | Train acc: 0.8442 | Val acc: 0.5890
Epoch 26/30 | Train acc: 0.8472 | Val acc: 0.5790
Epoch 27/30 | Train acc: 0.8502 | Val acc: 0.5840
Epoch 28/30 | Train acc: 0.8634 | Val acc: 0.5790
Epoch 29/30 | Train acc: 0.8770 | Val acc: 0.5740
Epoch 30/30 | Train acc: 0.8764 | Val acc: 0.5910
Plot saved to results.png

Final test accuracy: 59.10%

```

Figure 17. Final animal-class terminal run reaching **59.10%** final test accuracy.

Optional extra evidence:

```
Training on: cuda
Epoch 1/30 | Train acc: 0.1040 | Val acc: 0.1000
Epoch 2/30 | Train acc: 0.1038 | Val acc: 0.1230
Epoch 3/30 | Train acc: 0.1180 | Val acc: 0.1090
Epoch 4/30 | Train acc: 0.1240 | Val acc: 0.1290
Epoch 5/30 | Train acc: 0.1286 | Val acc: 0.1720
Epoch 6/30 | Train acc: 0.1346 | Val acc: 0.1620
Epoch 7/30 | Train acc: 0.1382 | Val acc: 0.1960
Epoch 8/30 | Train acc: 0.1312 | Val acc: 0.1690
Epoch 9/30 | Train acc: 0.1466 | Val acc: 0.2010
Epoch 10/30 | Train acc: 0.1326 | Val acc: 0.1800
Epoch 11/30 | Train acc: 0.1410 | Val acc: 0.1930
Epoch 12/30 | Train acc: 0.1458 | Val acc: 0.1970
Epoch 13/30 | Train acc: 0.1644 | Val acc: 0.1870
Epoch 14/30 | Train acc: 0.1690 | Val acc: 0.2000
Epoch 15/30 | Train acc: 0.1482 | Val acc: 0.1700
Epoch 16/30 | Train acc: 0.1582 | Val acc: 0.1850
Epoch 17/30 | Train acc: 0.1632 | Val acc: 0.2030
Epoch 18/30 | Train acc: 0.1666 | Val acc: 0.1720
Epoch 19/30 | Train acc: 0.1582 | Val acc: 0.1990
Epoch 20/30 | Train acc: 0.1460 | Val acc: 0.1700
Epoch 21/30 | Train acc: 0.1658 | Val acc: 0.2110
Epoch 22/30 | Train acc: 0.1786 | Val acc: 0.2160
Epoch 23/30 | Train acc: 0.1754 | Val acc: 0.2140
Epoch 24/30 | Train acc: 0.1772 | Val acc: 0.2150
Epoch 25/30 | Train acc: 0.1792 | Val acc: 0.1000
Epoch 26/30 | Train acc: 0.1094 | Val acc: 0.1100
Epoch 27/30 | Train acc: 0.1182 | Val acc: 0.1750
Epoch 28/30 | Train acc: 0.1142 | Val acc: 0.1600
Epoch 29/30 | Train acc: 0.1164 | Val acc: 0.1720
Epoch 30/30 | Train acc: 0.1236 | Val acc: 0.1550
Plot saved to results.png

Final test accuracy: 15.50%
```

Optional Figure. Poor local terminal run reaching **15.50%**, suggesting an unsuccessful configuration or training issue.

```

Training on: c000
Epoch 1/30 | Train acc: 0.0948 | Val acc: 0.1280
Epoch 2/30 | Train acc: 0.1690 | Val acc: 0.2020
Epoch 3/30 | Train acc: 0.2324 | Val acc: 0.2670
Epoch 4/30 | Train acc: 0.2884 | Val acc: 0.3340
Epoch 5/30 | Train acc: 0.3172 | Val acc: 0.3610
Epoch 6/30 | Train acc: 0.3306 | Val acc: 0.3750
Epoch 7/30 | Train acc: 0.3626 | Val acc: 0.4010
Epoch 8/30 | Train acc: 0.4020 | Val acc: 0.4380
Epoch 9/30 | Train acc: 0.4292 | Val acc: 0.4170
Epoch 10/30 | Train acc: 0.4474 | Val acc: 0.4660
Epoch 11/30 | Train acc: 0.4802 | Val acc: 0.4580
Epoch 12/30 | Train acc: 0.5036 | Val acc: 0.5010
Epoch 13/30 | Train acc: 0.5374 | Val acc: 0.5030
Epoch 14/30 | Train acc: 0.5594 | Val acc: 0.5020
Epoch 15/30 | Train acc: 0.5770 | Val acc: 0.5360
Epoch 16/30 | Train acc: 0.6008 | Val acc: 0.5300
Epoch 17/30 | Train acc: 0.6140 | Val acc: 0.5500
Epoch 18/30 | Train acc: 0.6444 | Val acc: 0.5650
Epoch 19/30 | Train acc: 0.6672 | Val acc: 0.5390
Epoch 20/30 | Train acc: 0.6754 | Val acc: 0.5570
Epoch 21/30 | Train acc: 0.6718 | Val acc: 0.5570
Epoch 22/30 | Train acc: 0.7148 | Val acc: 0.5600
Epoch 23/30 | Train acc: 0.7206 | Val acc: 0.5450
Epoch 24/30 | Train acc: 0.7388 | Val acc: 0.5670
Epoch 25/30 | Train acc: 0.7590 | Val acc: 0.5700
Epoch 26/30 | Train acc: 0.7672 | Val acc: 0.5830
Epoch 27/30 | Train acc: 0.7710 | Val acc: 0.5820
Epoch 28/30 | Train acc: 0.7956 | Val acc: 0.5700
Epoch 29/30 | Train acc: 0.8080 | Val acc: 0.5780
Epoch 30/30 | Train acc: 0.8072 | Val acc: 0.5720
Plot saved to results.png

Final test accuracy: 57.20%

```

Optional Figure. Additional local terminal run reaching **57.20%** final test accuracy.

```
Training on 100000
Epoch 1/30 | Train acc: 0.1078 | Val acc: 0.1000
Epoch 2/30 | Train acc: 0.1012 | Val acc: 0.1600
Epoch 3/30 | Train acc: 0.2160 | Val acc: 0.2990
Epoch 4/30 | Train acc: 0.3046 | Val acc: 0.3650
Epoch 5/30 | Train acc: 0.3834 | Val acc: 0.4550
Epoch 6/30 | Train acc: 0.4292 | Val acc: 0.5140
Epoch 7/30 | Train acc: 0.4570 | Val acc: 0.5280
Epoch 8/30 | Train acc: 0.4984 | Val acc: 0.5380
Epoch 9/30 | Train acc: 0.5150 | Val acc: 0.5500
Epoch 10/30 | Train acc: 0.5344 | Val acc: 0.5400
Epoch 11/30 | Train acc: 0.5486 | Val acc: 0.5850
Epoch 12/30 | Train acc: 0.5806 | Val acc: 0.5760
Epoch 13/30 | Train acc: 0.5844 | Val acc: 0.5860
Epoch 14/30 | Train acc: 0.6030 | Val acc: 0.5710
Epoch 15/30 | Train acc: 0.6134 | Val acc: 0.5620
Epoch 16/30 | Train acc: 0.6368 | Val acc: 0.5770
Epoch 17/30 | Train acc: 0.6540 | Val acc: 0.5890
Epoch 18/30 | Train acc: 0.6646 | Val acc: 0.5980
Epoch 19/30 | Train acc: 0.6776 | Val acc: 0.5960
Epoch 20/30 | Train acc: 0.6906 | Val acc: 0.6010
Epoch 21/30 | Train acc: 0.7166 | Val acc: 0.5970
Epoch 22/30 | Train acc: 0.7100 | Val acc: 0.5870
Epoch 23/30 | Train acc: 0.7334 | Val acc: 0.5920
Epoch 24/30 | Train acc: 0.7458 | Val acc: 0.5950
Epoch 25/30 | Train acc: 0.7608 | Val acc: 0.6030
Epoch 26/30 | Train acc: 0.7728 | Val acc: 0.6050
Epoch 27/30 | Train acc: 0.7844 | Val acc: 0.6030
Epoch 28/30 | Train acc: 0.7910 | Val acc: 0.6100
Epoch 29/30 | Train acc: 0.8018 | Val acc: 0.6260
Epoch 30/30 | Train acc: 0.8100 | Val acc: 0.6070
Final test accuracy: 60.70%
```

Optional Figure. Additional local terminal run reaching **60.70%** final test accuracy.

```

Training on: cuda
Epoch 1/30 | Train acc: 0.0932 | Val acc: 0.1000
Epoch 2/30 | Train acc: 0.0966 | Val acc: 0.1000
Epoch 3/30 | Train acc: 0.0958 | Val acc: 0.1000
Epoch 4/30 | Train acc: 0.0996 | Val acc: 0.1000
Epoch 5/30 | Train acc: 0.1000 | Val acc: 0.1000
Epoch 6/30 | Train acc: 0.1054 | Val acc: 0.1010
Epoch 7/30 | Train acc: 0.1020 | Val acc: 0.1000
Epoch 8/30 | Train acc: 0.0944 | Val acc: 0.1640
Epoch 9/30 | Train acc: 0.1212 | Val acc: 0.1340
Epoch 10/30 | Train acc: 0.1400 | Val acc: 0.1370
Epoch 11/30 | Train acc: 0.1790 | Val acc: 0.2570
Epoch 12/30 | Train acc: 0.2372 | Val acc: 0.2870
Epoch 13/30 | Train acc: 0.2914 | Val acc: 0.3450
Epoch 14/30 | Train acc: 0.3182 | Val acc: 0.3530
Epoch 15/30 | Train acc: 0.3556 | Val acc: 0.3870
Epoch 16/30 | Train acc: 0.3700 | Val acc: 0.4260
Epoch 17/30 | Train acc: 0.4038 | Val acc: 0.4210
Epoch 18/30 | Train acc: 0.4236 | Val acc: 0.4320
Epoch 19/30 | Train acc: 0.4462 | Val acc: 0.4480
Epoch 20/30 | Train acc: 0.4490 | Val acc: 0.4820
Epoch 21/30 | Train acc: 0.4804 | Val acc: 0.4750
Epoch 22/30 | Train acc: 0.4990 | Val acc: 0.5000
Epoch 23/30 | Train acc: 0.5152 | Val acc: 0.5290
Epoch 24/30 | Train acc: 0.5364 | Val acc: 0.5350
Epoch 25/30 | Train acc: 0.5562 | Val acc: 0.5540
Epoch 26/30 | Train acc: 0.5778 | Val acc: 0.5690
Epoch 27/30 | Train acc: 0.5950 | Val acc: 0.5640
Epoch 28/30 | Train acc: 0.6136 | Val acc: 0.5700
Epoch 29/30 | Train acc: 0.6224 | Val acc: 0.5720
Epoch 30/30 | Train acc: 0.6406 | Val acc: 0.5930
Final test accuracy: 59.30%
(gpu) test02: (loss: 1.0000)

```

Optional Figure. Additional local terminal run reaching **59.30%** final test accuracy.

8. Terminal run results

Several local terminal runs were captured in the screenshots. The visible final test accuracies are summarized below.

Run shown in terminal screenshots	Visible configuration / context	Final test accuracy
Baseline-style run	30 epochs, CUDA	61.50%
Additional baseline-style run	30 epochs, CUDA	60.50%
Deeper / 50-epoch run	50 epochs, CUDA	54.90%
LeakyReLU + Adam run	30 epochs, CUDA	67.30%

Run shown in terminal screenshots	Visible configuration / context	Final test accuracy
SGD run	30 epochs, CUDA	55.30%
Extended SGD run	80 epochs, CUDA	59.20%
Additional animal-class run	30 epochs, CUDA	59.70%
Tuned run	30 epochs, CUDA	68.00%
Extended tuned run	50 epochs, CUDA	69.90%
Shorter tuned run	15 epochs, CUDA	65.80%
Final animal-class run	30 epochs, CUDA	59.10%
Failed / poor run	30 epochs, CUDA	15.50%
Additional run	30 epochs, CUDA	57.20%
Additional run	30 epochs, CUDA	60.70%
Additional run	30 epochs, CUDA	59.30%

The strongest visible local terminal result was **69.90% final test accuracy**, achieved by an extended tuned run. The next strongest visible result was **68.00%**, followed by **67.30%** from the LeakyReLU + Adam run.

The lowest visible result was **15.50%**, which suggests that one run failed to train properly or used a poor configuration. Since most other runs reached around 55–70%, this result should be treated as an unsuccessful run rather than representative model performance.

9. Result interpretation

The local terminal runs confirm that the CIFAR-100 CNN workflow could be executed outside a notebook environment. The repeated `Training on: cuda` output shows that PyTorch used GPU acceleration for the terminal runs.

The saved output images show what the script produced during execution. The sample image grid confirms that the animal-class subset was loaded correctly. The training curve plots show that the model learned from the training data, but also that validation accuracy often plateaued while training accuracy continued to rise. This indicates overfitting, especially in the animal-class runs.

The results varied noticeably between runs. Final test accuracy ranged from **15.50%** to **69.90%**, although the **15.50%** run appears to be an unsuccessful or poorly configured run. Most successful runs were in the range of approximately **55% to 70%**.

The baseline-style runs reached **61.50%** and **60.50%**. The deeper 50-epoch run reached only **54.90%**, showing that increasing depth or training longer did not automatically improve performance. The LeakyReLU + Adam run improved to **67.30%**, which supports the earlier observation that LeakyReLU helped the model train better.

The SGD optimizer runs reached **55.30%** after 30 epochs and **59.20%** after 80 epochs. This shows that SGD improved with longer training but still did not outperform the stronger Adam-based runs.

The best visible local terminal result was **69.90%**, achieved by an extended tuned run. This was close to the strongest notebook results and shows that the local terminal setup was capable of producing competitive results when the configuration was suitable.

10. Conclusion

The local VS Code / terminal experiment successfully ran the CIFAR-100 CNN workflow as a standalone PyTorch script. The terminal output confirmed GPU acceleration through `Training on: cuda`, and the script saved result plots and prediction images to files.

Across the visible terminal runs, the best final test accuracy was **69.90%**. Other strong results included **68.00%** and **67.30%**, while weaker runs such as **54.90%**, **55.30%**, and **15.50%** showed that not every configuration trained successfully.

Overall, the terminal results support the same conclusion as the notebook experiments: CNN performance on CIFAR-100 depends strongly on model configuration, activation function, optimizer, batch size, training duration, and selected classes. The best local terminal result came from a tuned run, while deeper models or SGD-based configurations did not automatically improve performance.